

Green Pace Security Policy

Project Two: Security Policy Presentation

Crystal Schippers

CS 405

August 16, 2026

Video Link: <https://youtu.be/p0fSkHRUcrg>

Slide 1 – Title Page

Hello, my name is Crystal Schippers. This presentation introduces the Green Pace Security Policy and explains how secure coding, architectural controls, encryption, Triple-A, unit testing, and automation work together as a defense-in-depth strategy.

Slide 2 – Security Policy Overview

The Green Pace Security Policy standardizes secure development practices across the organization. The policy applies to staff who create, deploy, or support custom software. Its purpose is to make security repeatable and consistent rather than dependent on individual developer habits. Defense in depth is the foundation, so multiple controls are used throughout the development lifecycle.

Slide 3 – Threat Matrix & Attack Surface

The threat matrix helps Green Pace prioritize vulnerabilities according to their potential impact and likelihood. SQL injection and memory-safety problems receive high attention because they can lead to unauthorized data access, corruption, or system compromise. The attack surface is reduced by validating input, sanitizing data, using least privilege, default deny, secure coding standards, and testing.

Slide 4 – Automation & External Testing

Automation allows security checks to happen repeatedly and consistently. Visual Studio warnings and code analysis provide feedback during development and builds. Cppcheck provides an independent static-analysis perspective. Unit tests provide another layer by verifying expected behavior and expected failures. Using several tools supports defense in depth because one tool may identify an issue another tool does not.

Slide 5 – Ten Security Principles

The policy uses ten guiding principles: validate input data; heed compiler warnings; architect and design for security policies; keep it simple; default deny; least privilege; sanitize data sent to other systems; practice defense in depth; use effective quality assurance; and adopt a secure coding standard. Together, these principles create a consistent foundation for secure development.

Slide 6 – Principles Mapped to Coding Standards

The principles are mapped to the coding standards so developers can see why each rule exists. Input validation and sanitization directly support the string, SQL injection, and character-and-string standards. Compiler warnings and quality assurance support safe data conversion, assertions, exceptions, and stream handling. Defense in depth applies across all standards because Green Pace should not depend on one security mechanism.

Slide 7 – Coding Standards: Priority Order

The coding standards are prioritized according to security impact and exploitability. SQL injection and memory-safety issues receive the greatest attention because they can directly affect confidentiality and system integrity. String and integer conversion problems follow because incorrect data handling can create vulnerabilities or unexpected behavior. The remaining standards address reliability and undefined behavior that can weaken security.

Slide 8 – Encryption Strategy

Green Pace uses three encryption perspectives: at rest, in flight, and in use. Data at rest is protected while stored. Data in flight is protected while moving between systems. Data in use addresses protection while information is actively processed. Encryption is important for confidentiality, but it must work with authentication, authorization, and other defense-in-depth controls.

Slide 9 – Triple-A Framework

Triple-A consists of authentication, authorization, and accounting. Authentication verifies identity. Authorization determines what that identity is allowed to do. Accounting records activity so actions can be reviewed and investigated. Green Pace's policy specifically considers user logins, database changes, new users, access levels, and files accessed. Existing operating-system, firewall, and anti-malware logs provide part of this accounting layer.

Slide 10 – Unit Test: CollectionSmartPointerIsNotNull

This is a positive test. Confirms the fixture creates a valid smart pointer and the collection address is not null. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 11 – Unit Test: IsEmptyOnCreate

This is a positive test. Confirms a newly created vector contains zero elements. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 12 – Unit Test: CanAddToEmptyCollection

This is a positive test. Adds one element and verifies the collection becomes nonempty with size one. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 13 – Unit Test: CanAddFiveValuesToCollection

This is a positive test. Adds five elements and verifies the vector size becomes five. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 14 – Unit Test: MaxSizeGreaterThanSize (0)

This is a positive test. This parameterized test uses the value in the test name to verify that a vector's `max_size` is greater than its current size. It demonstrates how one test definition can be executed with multiple inputs while maintaining the same security and correctness condition. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 15 – Unit Test: MaxSizeGreaterThanSize (1)

This is a positive test. This parameterized test uses the value in the test name to verify that a vector's `max_size` is greater than its current size. It demonstrates how one test definition can be executed with multiple inputs while maintaining the same security and correctness condition. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 16 – Unit Test: MaxSizeGreaterThanSize (5)

This is a positive test. This parameterized test uses the value in the test name to verify that a vector's `max_size` is greater than its current size. It demonstrates how one test definition can be executed with multiple inputs while maintaining the same security and correctness condition. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 17 – Unit Test: MaxSizeGreaterThanSize (10)

This is a positive test. This parameterized test uses the value in the test name to verify that a vector's `max_size` is greater than its current size. It demonstrates how one test definition can be executed with multiple inputs while maintaining the same security and correctness condition. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 18 – Unit Test: CapacityGreaterThanOrEqualToSize (0)

This is a positive test. This parameterized test verifies that vector capacity is at least its current size for the specified input. Parameterization reduces duplicate test code while expanding coverage. The

test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 19 – Unit Test: CapacityGreaterThanOrEqualToSize (1)

This is a positive test. This parameterized test verifies that vector capacity is at least its current size for the specified input. Parameterization reduces duplicate test code while expanding coverage. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 20 – Unit Test: CapacityGreaterThanOrEqualToSize (5)

This is a positive test. This parameterized test verifies that vector capacity is at least its current size for the specified input. Parameterization reduces duplicate test code while expanding coverage. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 21 – Unit Test: CapacityGreaterThanOrEqualToSize (10)

This is a positive test. This parameterized test verifies that vector capacity is at least its current size for the specified input. Parameterization reduces duplicate test code while expanding coverage. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 22 – Unit Test: ResizeIncreasesCollectionSize

This is a positive test. Resizes an empty vector to five elements and verifies the new size. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 23 – Unit Test: ResizeDecreasesCollectionSize

This is a positive test. Adds five elements, resizes to two, and verifies the reduced size. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 24 – Unit Test: ResizeCollectionSizeToZero

This is a positive test. Adds five elements, resizes to zero, and verifies the vector is empty. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 25 – Unit Test: ClearCollection

This is a positive test. Clears a populated vector and verifies both empty state and zero size. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 26 – Unit Test: EraseCollection

This is a positive test. Erases the entire begin-to-end range and verifies the vector is empty. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 27 – Unit Test: ReserveIncreaseCollectionCapacity

This is a positive test. Reserves capacity without changing size and verifies capacity increased. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 28 – Unit Test: OutOfRangeExceptionThrown

This is a negative test. This is a negative test. It intentionally accesses an invalid vector index and uses ASSERT_THROW to prove that std::out_of_range is generated. This demonstrates that expected failure behavior is tested instead of assuming invalid input will never occur. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 29 – Unit Test: AssignValuesToCollection

This is a positive test. Assigns five copies of value 10 and verifies both size and values. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 30 – Unit Test: IncreaseCollectionReserveAboveMaxSize

This is a negative test. This is a negative test. It attempts to reserve more than the vector's maximum supported size and verifies std::length_error. Testing this boundary helps ensure the program handles invalid resource requests predictably. The test uses Google Test assertions to explicitly prove the expected condition. Keeping this test in the automated suite allows future changes to be checked for regressions.

Slide 31 – Unit Testing Results

The completed Google Test suite ran 21 tests successfully. The positive tests demonstrate normal vector operations, while the two negative tests demonstrate that expected exceptions are correctly detected. This is important because secure software must prove not only that valid inputs work, but also that invalid conditions are handled safely.

Slide 32 – DevSecOps Automation Pipeline

Security automation should be distributed across the development pipeline. During planning, the team reviews threats and attack surface. During coding and building, compiler warnings and static analysis identify problems early. During testing, unit and security tests verify behavior. Before

release, security checks should act as quality gates. During operations, logs and monitoring provide feedback that can be used to improve future development.

Slide 33 – Risks and Benefits

Addressing vulnerabilities early costs developer time and may create some false positives, but it is generally less expensive than fixing a vulnerability after release. Automation also creates repeatable evidence for audits. Waiting can increase remediation costs and expose customers or company systems to preventable attacks. The best approach is to prioritize high-impact issues while continuously reducing the remaining risk.

Slide 34 – Recommendations & Future Gaps

My recommendations are to make security checks mandatory in pull requests and CI builds, track unresolved findings, expand security-focused negative testing, review dependencies and compiler settings, and regularly update threat models. The policy should also continue its annual review process and be updated when regulations, technology, or threats change. This keeps the policy relevant as Green Pace grows.

Slide 35 – Conclusion

The Green Pace policy transforms secure-development practices into repeatable standards. Defense in depth connects secure coding, encryption, Triple-A, static analysis, and unit testing. The most important lesson is that security should be built into the development lifecycle instead of being treated as a final step. Continuous testing and automation make that approach practical and scalable.

Slide 36 – References

The presentation is supported by the SEI CERT C and C++ coding standards, Cppcheck documentation, Microsoft Visual Studio Code Analysis documentation, and SonarSource documentation. These references provide the secure-coding and automated-analysis foundation used by the Green Pace policy.

References

Cppcheck. (n.d.). *Cppcheck: A tool for static C/C++ code analysis*.

<https://cppcheck.sourceforge.io/>

Microsoft. (n.d.). *Code analysis for C/C++ overview*. Microsoft Learn.

<https://learn.microsoft.com/en-us/visualstudio/code-quality/>

Software Engineering Institute. (n.d.). *SEI CERT C coding standard*. Carnegie Mellon

University. <https://wiki.sei.cmu.edu/confluence/display/c>

Software Engineering Institute. (n.d.). *SEI CERT C++ coding standard*. Carnegie Mellon

University. <https://wiki.sei.cmu.edu/confluence/display/cplusplus>

SonarSource. (n.d.). *C/C++ analysis*. SonarQube documentation.

<https://docs.sonarsource.com/sonarqube-server/analyzing-source-code/languages/c-family/>